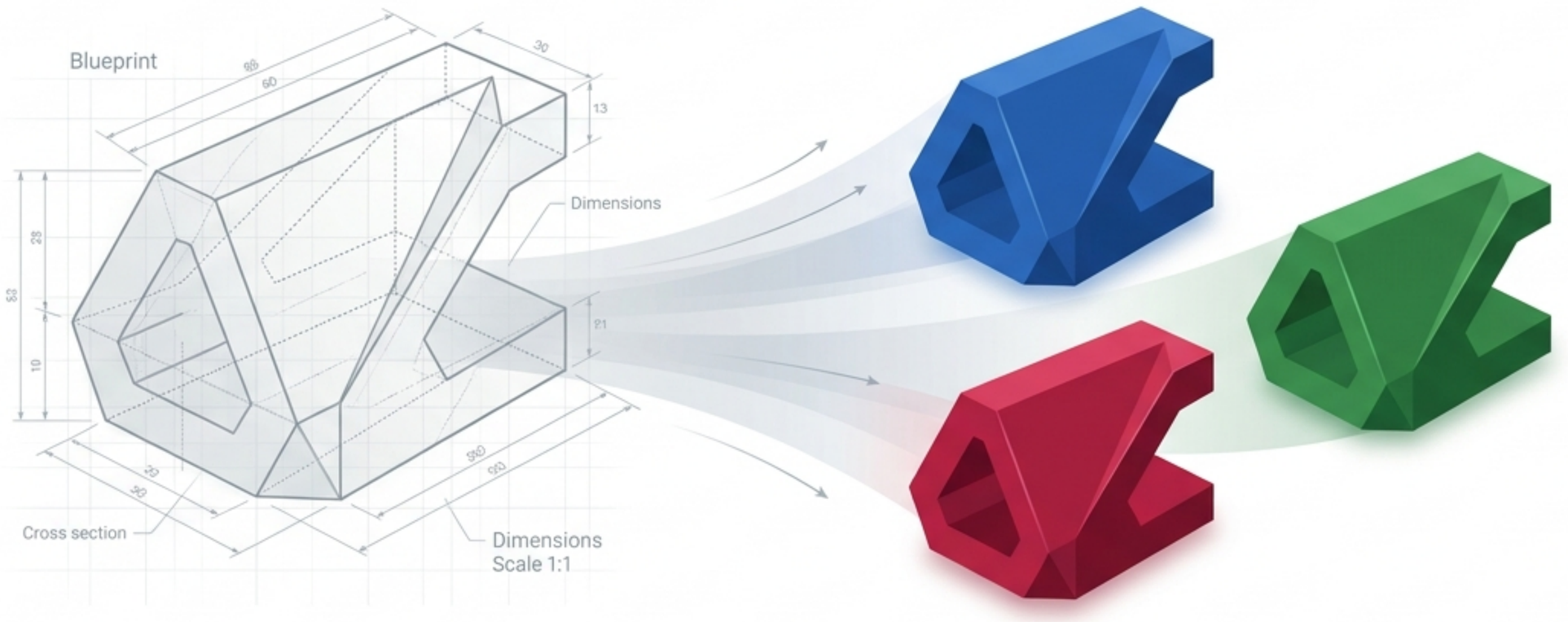
Aula 5: Classes e Objetos

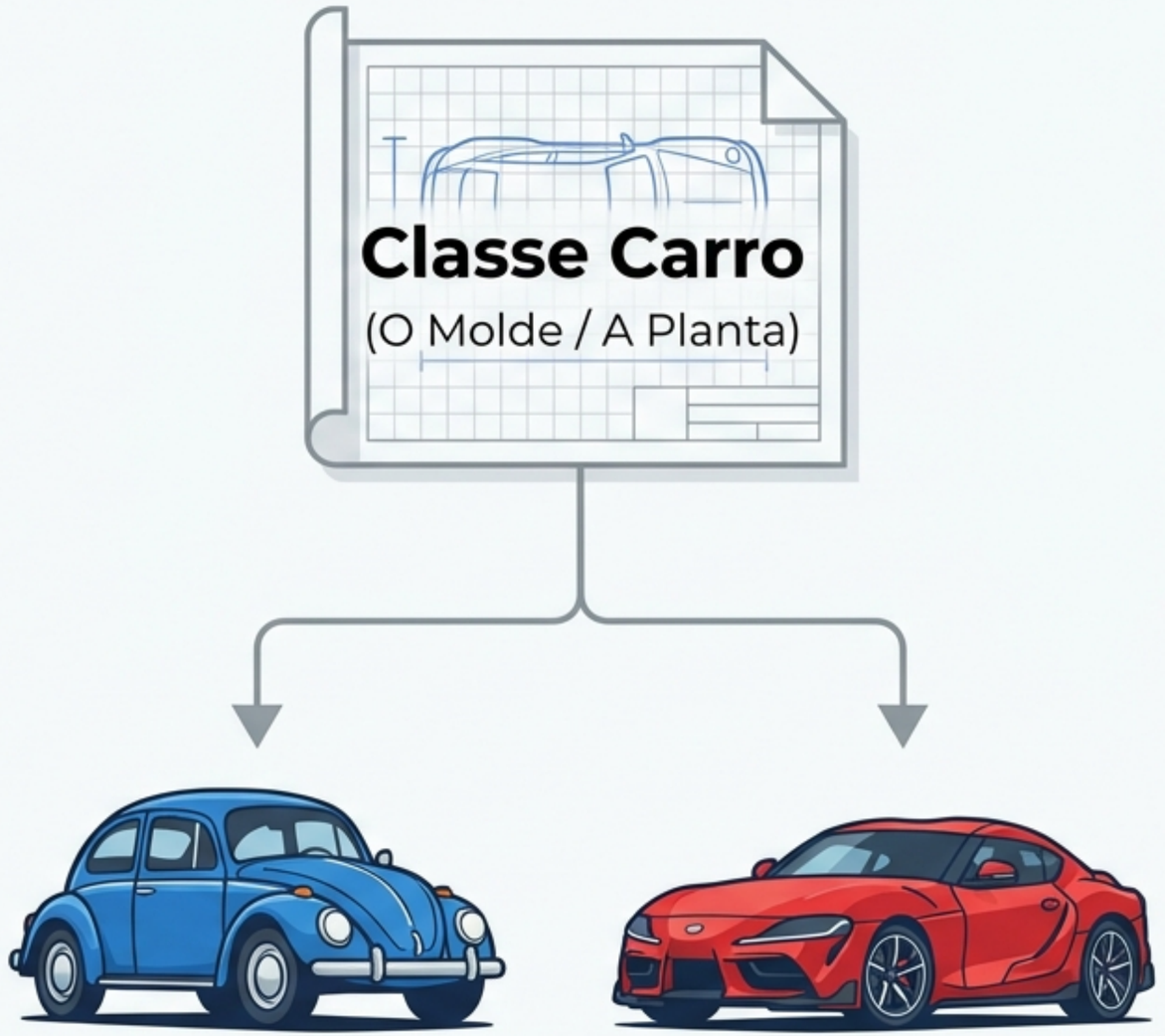
Prof. Ronaldo Borges



Gemini Notebook

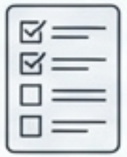
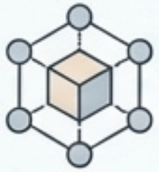
O Paradigma de Orientação a Objetos

A OOP é baseada na abstração do mundo real. Nós agrupamos dados (características) e comportamentos (ações) em uma única unidade lógica chamada Objeto.



Gemini Notebook

Estruturado vs. Orientado a Objetos

	 **Estruturada**	 **Orientada a Objetos** **Objetos**
O Foco	Foco em "Como fazer" (Procedimentos e Funções sequenciais).	Foco em "Quem faz" (Atores e Objetos que interagem).
Organização	Dados e funções vivem separados.	Dados e funções andam juntos (Encapsulamento).
Unidade Básica	A Função.	A Classe / O Objeto.

Gemini Notebook

Objetos Literais em JavaScript

Chave
(Propriedade/Atributo)

```
const usuario = {
  nome: 'João',
  idade: 30
};
```

A sintaxe literal usa chaves para criaes o objeto na memória.

Valor
(Dado armazenado)



Gemini Notebook

Acesso e Manipulação de Propriedades



Notação de Ponto

Mais comum, **limpa e direta**. Usada quando sabemos o nome exato da propriedade.

```
console.log(usuario.nome);
usuario.idade = 31;
```



Notação de Colchetes

Usada com chaves dinâmicas, variáveis, ou nomes com **espaços/caracteres** especiais.

```
const chaveDinamica = 'nome';
console.log(usuario[chaveDinamica]);
```

Gemini Notebook

Adicionando Comportamentos e o contexto this

```
const usuario = {
  nome: 'João',
  saudacao() {
    return `Olá, meu nome é ${this.nome}`;
  }
};

usuario.saudacao();
```

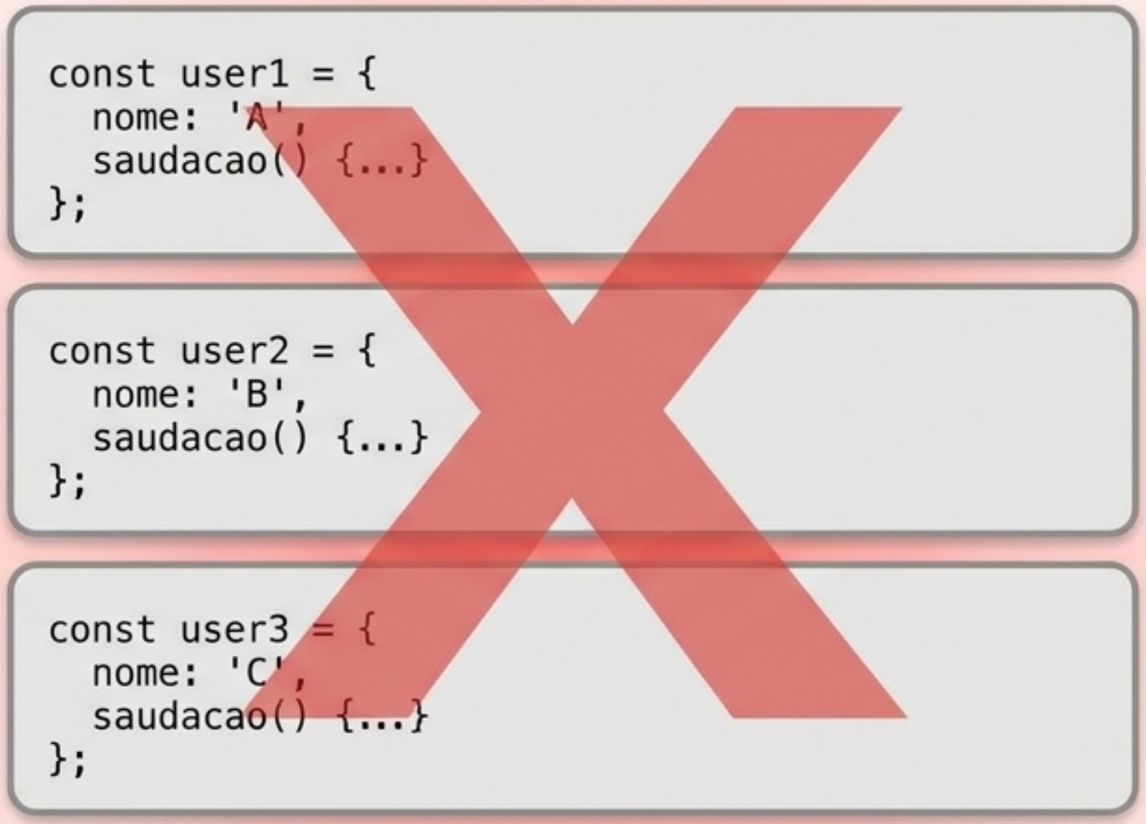


O Segredo do this

O **this** é o **próprio objeto**! Ele diz à função: **'Use a propriedade nome do objeto que me chamou'**. Sem o **this**, o JavaScript procuraria uma variável solta chamada nome fora do objeto.

Gemini Notebook

O Problema da Escala ⚠️



Copiar e colar literais viola princípios de programação (DRY - Don't Repeat Yourself) e consome o triplo de memória, pois a função saudacao() é recriada do zero para cada usuário.

Gemini Notebook

A Solução Clássica: Funções Construtoras

Antes das Classes existirem no JavaScript, usávamos funções normais como “fábricas” de objetos. **A convenção é usar a primeira letra Maiúscula.**

O operador **new** cria um **objeto vazio {}** e aponta o **this** para ele.

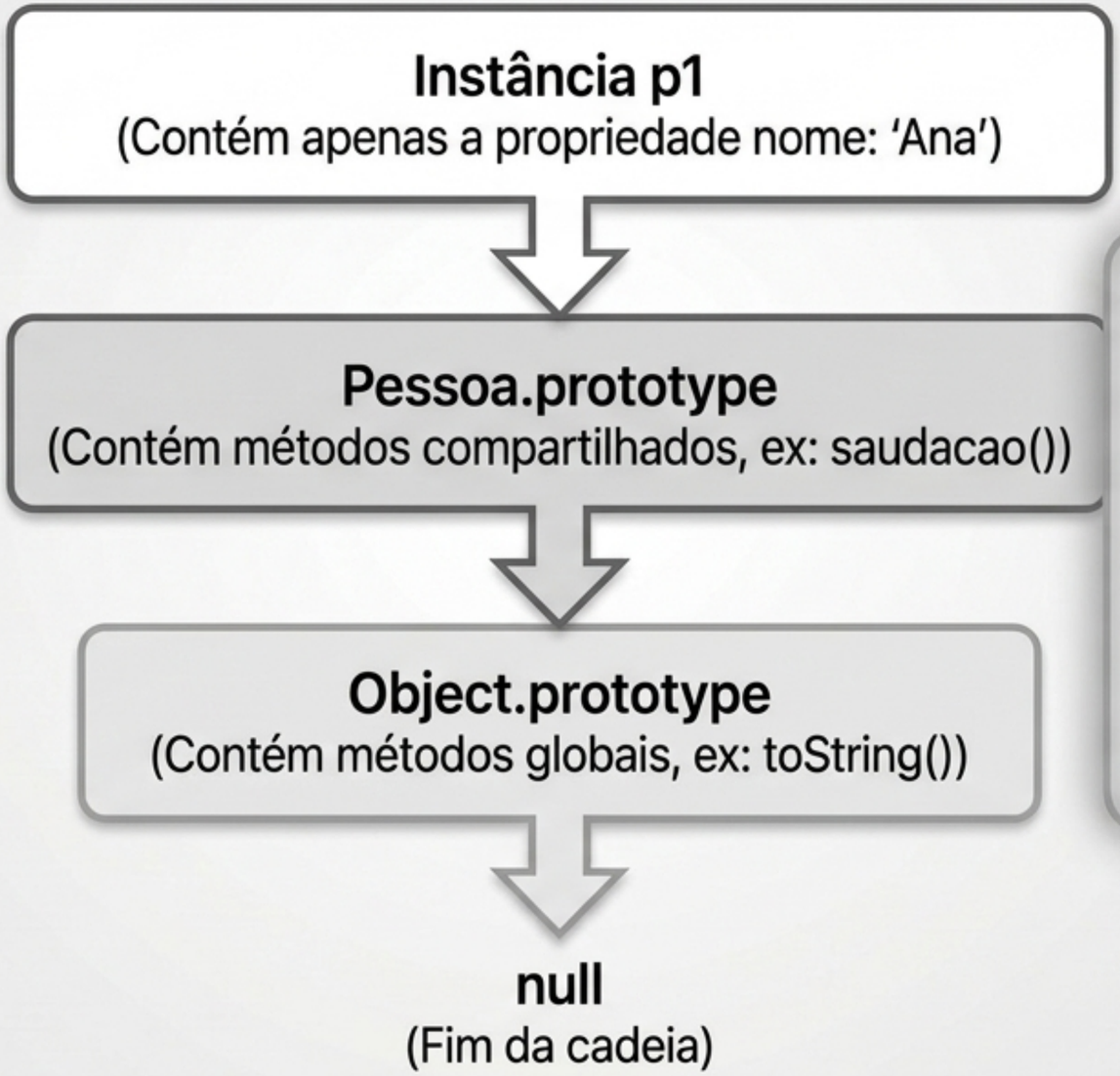
```
function Pessoa(nomeDesejado) {
        this.nome = nomeDesejado;
}

const p1 = new Pessoa('Ana');
```

Letra maiúscula indica que é um **construtor.**

Gemini Notebook

O Motor Oculto: A Cadeia de Protótipos



Prototype Chain: Se o JavaScript não encontra uma propriedade/método próprio objeto, ele "delega" a busca e sobe a cadeia de protótipos até encontrar ou chegar em null.
Isso economiza memória!

Gemini Notebook

A Evolução: O “Açúcar Sintático” do ES6

O Clássico

```
function Usuario(nome) {
  this.nome = nome;
}

Usuario.prototype.login =
function() { ... }
```

O Moderno (ES6)

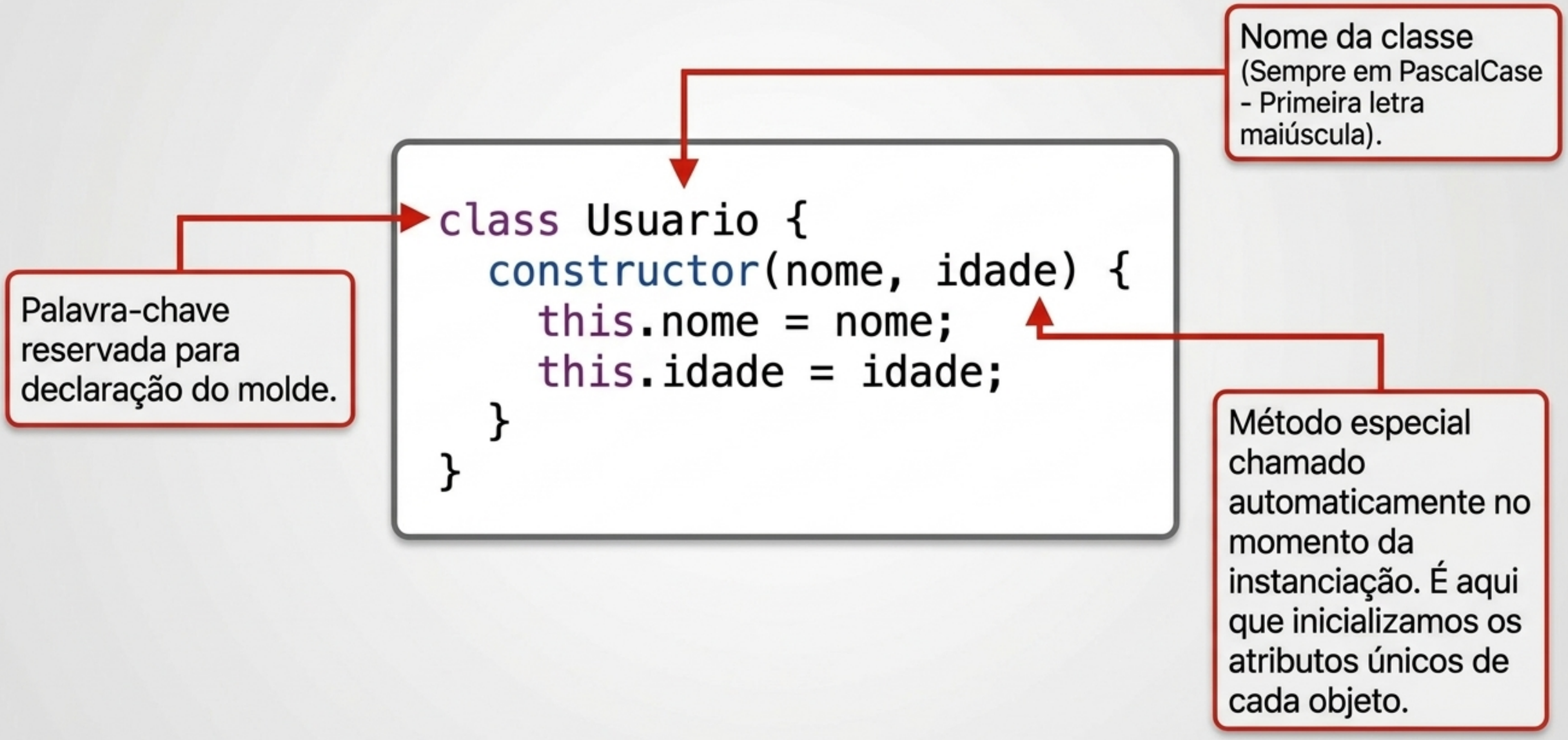
```
class Usuario {
  constructor(nome) {
    this.nome = nome;
  }

  login() { ... }
}
```

A introdução da palavra **class** no ECMAScript 2015 unificou a escrita. Por baixo dos dos panos, o JavaScript AINDA usa Funções Construtoras e Protótipos. É a mesma mecânica, mas com uma sintaxe mais limpa e amigável.

Gemini Notebook

Anatomia de uma Classe em JavaScript

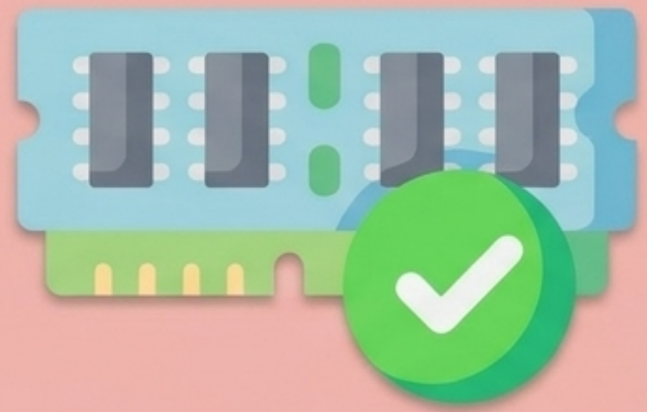


Gemini Notebook

Métodos de Instância na Classe

```
class Usuario {
  constructor(nome) {
    this.nome = nome;
  }

  // Método de Instância
  login() {
    console.log(`${this.nome}
efetuou login.`);
  }
}
```



Eficiência Automática

Qualquer método definido diretamente no corpo da classe (fora do construtor) é automaticamente colocado no prototype pelo JavaScript. Todas as instâncias compartilharão a mesma função na memória.

Gemini Notebook

Instanciação: Dando Vida ao Molde

```
const user1 = new Usuario('Ronaldo');
user1.login(); // Saída: Ronaldo efetuou login.
```



Classe **Usuario**

user1 ('Ronaldo') **user2** ('Ana') **user3** ('Carlos')

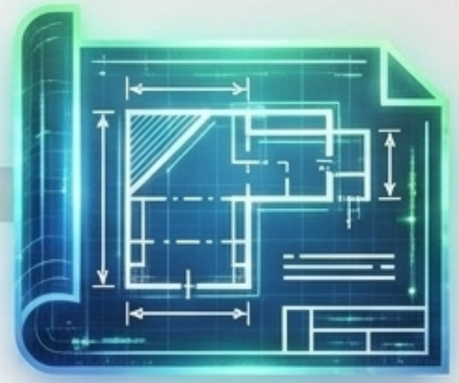
A classe é única, mas pode gerar infinitas instâncias. **Cada instância guarda seus próprios** dados de estado (**this.nome**), mas **divide os comportamentos (login())**.

Gemini Notebook

Síntese: A Jornada do Objeto em JS







Marco 1: Objeto Único

Objeto Literal ({ }). Rápido, mas não escala.

Marco 2: A Fábrica Clássica

Funções Construtoras (function) + prototype. Resolve a escala, mas a sintaxe é fragmentada.

Marco 3: O Molde Moderno

ES6 Classes (class + constructor). O padrão atual: elegante, familiar e eficiente.

Gemini Notebook

Referências Bibliográficas

BIFFI, K. B. JavaScript Básico ao Avançado. [S.l.: s.n.], [s.d.].

BORGES, R. Estruturas Sequenciais. IFPI - Campus Floriano, 2025. Material de aula.

GRONER, L. Estruturas de dados e algoritmos com JavaScript. São Paulo: Novatec, 2015.

ZAKAS, N. C. Princípios De Orientação A Objetos Em Javascript. São Paulo: Novatec, 2014.

Gemini Notebook